System Design Basics

Lesson 6: Reliability & Observability

Reliability & Observability - Study Notes

Key Concepts

• **Retry & Circuit Breaker Patterns** – mechanisms to handle transient failures and prevent cascading outages.

• **Observability** – the ability to understand the internal state of a system through logs, metrics, and traces.

• **Health Checks & Prometheus** – lightweight probes that expose service health, integrated with Prometheus for monitoring and alerting.

Summary

Reliability engineering focuses on making services resilient to failures. The retry pattern automatically re issues a request after a transient error, while the circuit breaker pattern stops traffic to a failing subsystem to allow it to recover. Together, they protect downstream services and keep the overall system stable.

Observability complements reliability by giving operators insight into why failures happen. Structured logging, distributed tracing, and metrics expose the system's behavior. Prometheus, a popular open source monitoring system, collects metrics from services and can trigger alerts based on health check endpoints. By combining retries, circuit breakers, and observability, teams can quickly detect, diagnose, and recover from issues.

Important Points

• **Retry Strategy**

- Use exponential back off with jitter to avoid thundering herd problems.
- Limit the number of attempts and set a maximum total timeout.
- Only retry on idempotent operations or when the error is transient (e.g., network hiccups).

• **Circuit Breaker**

- **Closed** – normal operation; counts failures.
- **Open** – immediately fails requests; prevents overload.
- **Half Open** – allows a limited number of requests to test recovery.
- Configure failure thresholds, timeout durations, and reset intervals.

• **Logging**

- Log at appropriate levels: `DEBUG` for detailed traces, `INFO` for normal operations, `WARN`/`ERROR` for failures.
- Use structured logs (JSON) so log aggregators can filter and query effectively.
- Include correlation IDs to trace a request across services.

• **Health Checks**

- **Liveness** – checks if the process is running (e.g., `/healthz` returns 200).
- **Readiness** – checks if the service can handle traffic (e.g., database connectivity).
- Expose metrics (`/metrics`) for Prometheus to scrape.

• **Prometheus Integration**

- Use client libraries (e.g., `prometheus_client` for Python, `micrometer` for Java) to expose counters, histograms, and gauges.
- Create alerts in Prometheus Alertmanager (e.g., `service_down`, `high_error_rate`).
- Visualize dashboards with Grafana.

Examples

1. Retry with Exponential Back off in Python

```python
import time
import random
import requests
from requests.exceptions import RequestException
def fetch_with_retry(url, retries=5, base_delay=0.1):
    for attempt in range(1, retries + 1):
        try:
            response = requests.get(url, timeout=2)
            response.raise_for_status()
            return response.json()
        except RequestException as e:
            if attempt == retries:
                raise
            # Exponential back  off with jitter
            delay = base_delay * (2 ** (attempt - 1)) + random.uniform(0, 0.1)
            time.sleep(delay)
# Usage

data = fetch_with_retry("https://api.example.com/resource")
```

Explanation: The function retries a GET request up to five times, doubling the delay each time and adding a small random jitter to avoid synchronized retries.

2. Circuit Breaker with `pybreaker` (Python)

```python
import pybreaker
import requests
breaker = pybreaker.CircuitBreaker(
```

```
    fail_max=3,          # Open after 3 consecutive failures
    reset_timeout=10,    # Wait 10 seconds before half  open
    name="ExampleService"
)
@breaker
def call_service(url):
    return requests.get(url, timeout=2).json()
```

When the service fails 3 times in a row, subsequent calls raise

CircuitBreakerError until the timeout expires.

```

*Explanation:* The decorator automatically tracks failures. Once the threshold is reached, the circuit opens, and further calls fail fast, allowing the downstream service to recover.

---

## Quick Review

1. What is the purpose of a circuit breaker in a distributed system?
2. How does exponential back  off with jitter improve retry reliability?
3. Distinguish between liveness and readiness health checks.
4. Which Prometheus metric type would you use to count the number of failed requests?
5. Why are structured logs preferred over plain text logs in microservices?

---

## Further Reading

- **Resilience Patterns** – “Patterns for Resilient Microservices” (Martin Fowler).
- **Distributed Tracing** – OpenTelemetry, Jaeger, Zipkin.
- **Prometheus & Grafana** – Official documentation, alerting rules, and dashboard templates.
- **Observability in Production** – “Observability Engineering” (Gartner).
- **Advanced Circuit Breaker Tuning** – Netflix Hystrix, Resilience4j.
```